

Tax AI Infrastructure

The plumbing behind the next three years of tax — and why the org chart, not the model, is the bottleneck.

A working thesis, grounded in data from in-house tax-technology teams.

THE QUESTION

In 2028, what part of the tax function will still be done **the way it is today?**

My answer: less than you think — but not for the reason the vendors are selling you.

WHERE THIS COMES FROM

Not vibes — a benchmark of real tax-tech functions

Anonymous

In-house tax & tax-technology teams report how they actually run: headcount, automation, architecture, AI stage.

Vendor-neutral

A non-profit, community-built dataset. No product to sell, no "magic quadrant" to defend.

Operational

9 dimensions — from build-vs-buy to data flow to regulatory response time — not survey-of-CFOs fluff.

Throughout this talk, the numbers come from what teams report about themselves — so we're calibrating to reality, not roadmaps.

Three shifts define the next 2-3 years

SHIFT 01

Software → Infrastructure

We stop buying "tax apps" and start assembling a stack: data, grounding, models, agents, governance.

SHIFT 02

Batch → Real-time

E-invoicing and continuous controls force tax from period-end batch to event-driven, always-on.

SHIFT 03

Headcount → Oversight

The job shifts from *doing* the work to *supervising* agents that do it. Trust becomes the product.

The winners won't have the best model. They'll have the best **infrastructure** around an ordinary one.

01

PART ONE

Where tax technology actually is today

Before we talk about the future, let's be honest about the floor we're starting from.

The tax stack is still mostly manual and batch

ERP + XLS

Still the dominant "tax data architecture" — no dedicated layer

Batch ETL

Data moves period-end, not in real time. Manual transfers are common.

Weeks

Typical time to respond to a regulatory change

You cannot bolt autonomous agents onto a spreadsheet you reconcile by hand once a quarter.

Most teams are at "exploration," not production

What's happening

- **Exploration & PoC** dominate the GenAI adoption stage.
- Use cases cluster in **research & drafting** — low-risk, human-checked.
- Few teams report **production** or enterprise-wide deployment.

Why it stalls

- No **data foundation** to ground the model in your facts.
- No **evaluation** harness → can't prove it's right → can't ship.
- Ownership is split across **Tax, Finance Systems, and IT**.

The gap between a demo and production **is the infrastructure**.
That's the whole game.

Three forcing functions are converging

1 · Capability

Frontier models now do multi-step reasoning, tool use, and long-context document work well enough for real tax tasks.

2 · Regulation

E-invoicing & continuous transaction controls (ViDA, national mandates) make tax data *real-time and machine-readable* by law.

3 · Pressure

Pillar Two, more jurisdictions, flat headcount. The work scales; the team doesn't. Something has to give.

Capability makes it **possible**. Regulation makes it **necessary**. Pressure makes it **urgent**.

02

PART TWO

The Tax AI Infrastructure stack

Six layers. You build them bottom-up. Skip one and the layer above it collapses.

From raw transactions to supervised autonomy

6

Governance & Evals

Accuracy tests, audit trail, explainability, human sign-off

5

Orchestration

Tools, ERP connectors, MCP, workflow & approvals

4

Agents

Research · provision · compliance · controversy

3

Models

Frontier foundation models + domain prompting/tuning

2

Knowledge & Grounding

Tax law, positions, prior filings — retrievable & cited

1

Data Foundation

Clean, real-time, lineage-tracked transaction & entity data

Read it bottom-up. Most failed AI pilots tried to start at layer 4.

If the data isn't real-time and trusted, stop here

- A **tax data layer** over the ERP — not 40 spreadsheets and a SharePoint.
- **Event-driven ingestion** so transactions arrive as they happen.
- **Lineage** — every number traces back to a source you can defend in an audit.
- Entity, jurisdiction, and product master data that's actually **maintained**.

1 **Data Foundation**

2 Knowledge

3 Models

4 Agents

This is unglamorous, expensive, and the single highest-ROI thing you can do for AI.

The model knows tax in general. It must know **your** tax.

What lives here

- Statutes, regs, rates — by jurisdiction, with effective dates.
- Your **tax positions**, memos, and prior-year filings.
- Rulings, e-invoicing schemas, and a jurisdiction knowledge graph.

Why it matters

- Retrieval grounds answers in **citable sources** — not memory.
- "Show me the authority" becomes a built-in, not a hope.
- This is what turns a chatbot into a **defensible** tool.

No grounding, no citations. No citations, no auditor will ever accept the output.

Models are the commodity. Don't fall in love with one.

- Frontier foundation models, swapped as they improve — assume **portability**.
- Most "tuning" you need is **prompting + grounding**, not training your own model.
- Right-size: a cheap fast model for extraction, a strong model for reasoning.
- Treat the model as a **replaceable part**, not the architecture.

↓ 10×

Rough order-of-magnitude drop in cost-per-task each generation. Plan for it.

The teams that hard-wired one vendor's model into everything will be re-platforming in 18 months.

From "ask a question" to "do the task"

Research agent

Reads the law + your positions, drafts a memo with citations and an explicit confidence flag.

Compliance agent

Prepares returns from the data layer, reconciles, flags anomalies for human review.

Provision agent

Pulls trial balance, computes the provision, drafts the workpapers and the narrative.

Controversy agent

Assembles audit responses from prior filings, source data, and the document trail.

An agent is a model **plus tools, plus a goal, plus guardrails** — it acts, it doesn't just chat.

Agents are only as useful as what they can reach

- **Connectors** to the ERP, the data layer, the filing portals, the tax engine.
- **MCP & tool standards** so any agent can call any system through one interface.
- **Workflow & approvals** — where a human signs, where an agent may proceed.
- **Observability** — logs of every action, ready for review and audit.

The integration tax

Benchmark data shows most teams are still on **vendor tax engines + ERP**, glued with batch jobs. Orchestration is where AI either plugs into reality — or stays a sandbox demo.

This is the layer that quietly decides whether your pilot ever touches a real return.

This is the moat — and the part everyone skips

Evaluate

A test set of known-answer tax cases. You measure accuracy *before* you trust output.
Regression-test every model swap.

Audit

Every answer carries its sources, inputs, and the reasoning trail.
Reproducible months later.

Control

Confidence thresholds route low-certainty items to humans. Clear sign-off and segregation of duties.

In tax, **being right is not enough — you must be able to prove you were right.** Governance is that proof.

03

PART THREE

Three stages of AI evolution in tax

AI-Native Foundation → Continuous Learning → Proactive Management.
The order is not optional.

How AI evolves through the tax function

STAGE 1 • 2026



AI-Native Foundation

- Data, grounding & AI built in — not bolted on
- Copilots assist; humans own every step
- You're laying track, not running trains yet

STAGE 2 • 2027



AI Continuous Learning

- System learns from your corrections & outcomes
- Adapts to new law, rates & positions automatically
- Accuracy compounds; agents run tasks for review

STAGE 3 • 2028



AI Proactive Management

- AI spots issues & acts *before* you ask
- Near-autonomous — **human approval retained**
- Tax team supervises; AI manages the operation

Each stage requires the one before it to be real. You earn the next stage with evidence — jurisdiction by jurisdiction.

This isn't a forecast — coding is already running the playbook

Software engineering is the same three stages, ~2-3 years ahead. It's the leading indicator for what reaches tax next.

STAGE 1 • DONE ✓

AI-Native Foundation

Writing code *with* AI is now the default. Tools like Claude Code and AI-first editors made the transformation — it's complete, not coming.

STAGE 2 • IN PROGRESS ↻

Continuous Learning

The strategic prize is now **data access** — the usage signal (accepts, edits, fixes) that lets the tools learn. That's why AI-native coding tools command a premium. Ongoing, not finished.

STAGE 3 • EMERGING 🔗

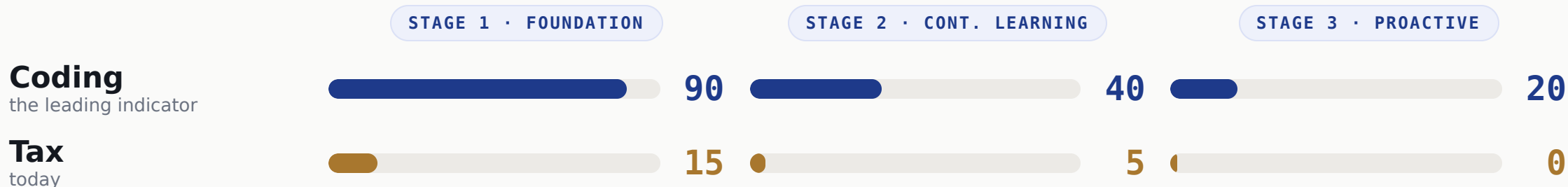
Proactive Management

Early signs — agents that proactively refactor, fix, open a PR. Real, but **not autonomous**: a human still approves the merge.

Tax is at Stage 1 today. Coding proves the sequence holds — and that the human-approval gate survives all the way through.

THE GAP, QUANTIFIED

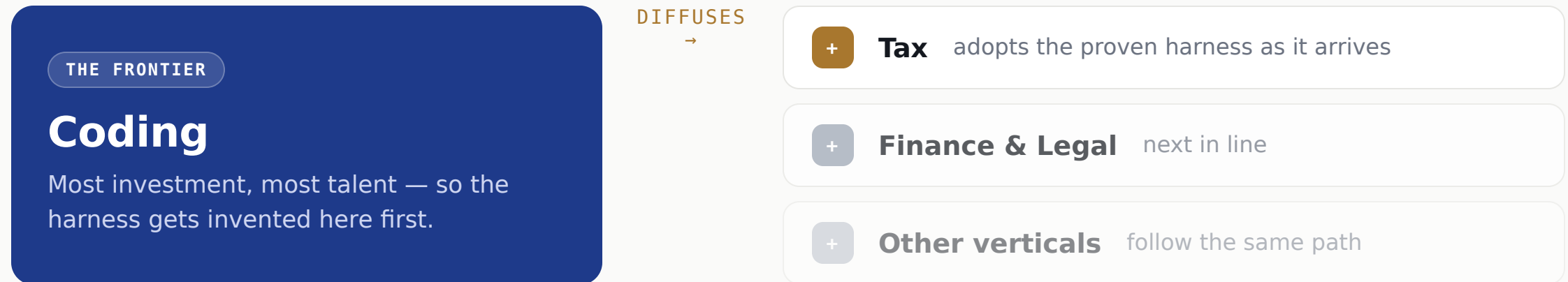
If we scored the three stages, 0-100



Same curve, ~2-3 years apart. Coding has finished Stage 1 and is mid-Stage 2 — **which is exactly the map tax gets to read ahead.**

The harness for Stage 2 & 3 doesn't exist yet — anywhere

Continuous Learning and Proactive Management need a **harness** — the scaffolding that makes models learn and act reliably. Nobody has built it. It's being figured out in coding, then it diffuses.



Even most AI startups are still at the **Foundation** stage. So you can't buy Stage 2 or 3 today — but you *can* build your foundation, and be ready the moment the harness diffuses.

Build the AI-Native Foundation across the whole tax lifecycle

You don't modernize one function in isolation. You build **one** AI-native foundation — and every stage of the tax lifecycle draws from it.

Tax Planning

Scenario modeling and structuring research, grounded in your own positions and the law.

Compliance & Reporting

High-volume, rules-based work on real-time data — the natural first mover.

Tax Audit

Controversy & defense built on the same lineage, sources, and document trail.

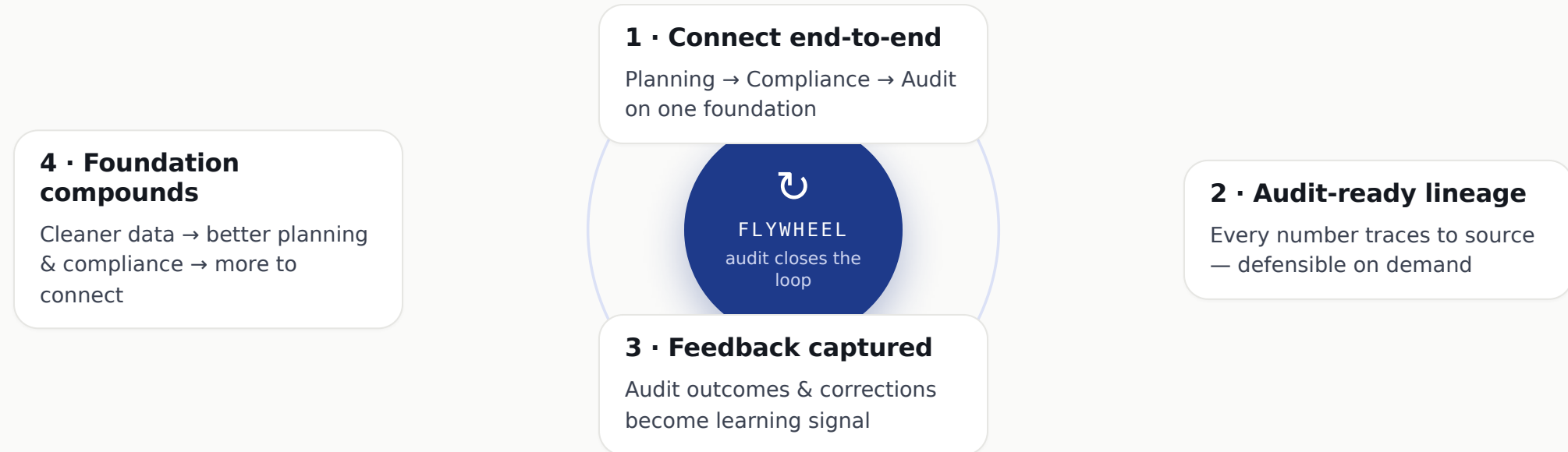
1

One AI-Native Foundation Shared data · grounding · governance — under all three

Foundation first. Build it once, build it well — then Continuous Learning and Proactive Management have somewhere to stand.

Connect the lifecycle end-to-end — a flywheel kicks in

Not a cost you pay before the payoff. Connect Planning → Compliance → Audit on one foundation and each loop compounds — **audit closes the loop.**



Each turn makes the data cleaner, the audit position stronger, the foundation smarter — **the exact feedback engine Stage 2 will run on.**

What an AI-native tax foundation actually requires

"Foundation" is not one thing you buy. It's nine infrastructure disciplines — each its own industry — and the work is **integrating all nine**.

Data platform / lakehouse

Snowflake · Databricks · BigQuery

Streaming & pipelines

Kafka/Confluent · Flink · Fivetran

Integration & connectors

SAP/Oracle · Vertex/Avalara · MuleSoft

Master data & catalog

Collibra · Alation · Atlan

Graph / knowledge graph

Neo4j · TigerGraph · Neptune

Retrieval / vector

Pinecone · Weaviate · pgvector

AI / ML platform

Bedrock · SageMaker · Vertex AI

Governance & security

Immuta · OneTrust · Okta

Eval & observability

LangSmith · Arize · Braintrust

Each box is its own engineering team in most companies. **The foundation is all nine, integrated** — that's the bar.

Why almost no tax function can stand this up

Messy, fragmented data

Tax data is scattered across ERPs, billing, supply chain, and spreadsheets — in many jurisdictions, with no clean source of truth.

Real-time is a different sport

Most teams live in batch ETL. Streaming, CDC, and event-driven infra are a specialized, expensive skill set they don't have.

Doubly-rare talent

You need people fluent in tax *and* platform/data/AI engineering. Each is scarce; the overlap barely exists.

It's a multi-year platform

Not a SKU you buy — a cross-cutting build that spans nine disciplines and years, not a quarter.

Split ownership

Tax vs. Finance Systems vs. IT. No single owner, no clear mandate, no budget line that fits.

Tax is a cost center

Funding hyperscale-grade infrastructure for a function seen as overhead is a hard sell to any CFO.

This is why "just add AI" stalls. The blocker isn't the model — it's **standing up nine infra disciplines a tax team was never built**

Why we were uniquely positioned to build it

Hyperscale data infra exists

One of the largest data platforms in the world is already here. Tax builds *on* it — we don't stand up a lakehouse from scratch.

Real-time by default

The company runs event-driven and streaming end-to-end. Tax data can be real-time native, not a batch afterthought.

World-class engineering

Deep bench of platform, data, and ML engineers — and a culture where tax-tech is embedded with real software teams.

AI platform & compute in-house

Frontier models, GPU scale, and mature ML tooling already operating — not a procurement exercise.

Scale justifies the build

Enormous transaction volume across many jurisdictions makes the ROI on a real foundation obvious.

Engineering-owned tax tech

The org model lets tax move like a product team — own the stack, ship, and iterate.

We didn't have to build the infra *and* the harness. The infra was already here — so we could put it to work for tax.

What gets automated first — and last

Automates early

- High-volume, rules-heavy **indirect tax / compliance**
- Data extraction & reconciliation
- First-draft research & documentation
- E-invoicing & transactional reporting

Automates last

- Novel **planning & structuring** judgment
- Controversy strategy & negotiation
- Anything with high ambiguity & high stakes
- Final sign-off & accountability — always human

Volume and structure get automated. **Judgment and accountability stay human** — and become more valuable.

04

PART FOUR

What it means for your team

Build-vs-buy, org design, talent, and the trust problem.

The line moves — buy the model, own the stack

Buy

Foundation models, tax engines, e-invoicing connectors, eval tooling. Commodities — don't build these.

Own

Your data foundation, your grounded knowledge, your evals, your governance. This is your moat.

Assemble

Orchestration glues bought parts to owned data. Increasingly low-code — but a real engineering discipline.

Benchmark reality: teams are split build/buy/hybrid — but the durable answer is **buy the commodity, own the context.**

The team composition inverts

- **Fewer preparers, more reviewers** — humans supervise agent output.
- A new role: the **tax engineer** — fluent in tax *and* data/AI.
- Ownership consolidates: AI forces Tax, Finance Systems & IT to **actually co-own** the stack.
- Skill mix shifts from spreadsheet ops to **data engineering & prompt/eval design**.

The benchmark signal

Teams already split tax-tech FTEs across frontend, backend, **data engineering**, and DevOps — and locate tax-tech in Tax, Finance Systems, or IT. The orgs that resolve that ownership question first will move fastest.

The scarce hire of 2028 isn't a tax expert or an engineer. It's the person who is **credibly both**.

What can go wrong — and how the stack answers it

Hallucinated authority

Model invents a citation. → **Grounding + evals** force every claim back to a real source.

Silent errors at scale

One bad rule, ten thousand returns. → **Confidence thresholds + sampling + audit trail.**

Regulator acceptance

"Explain this number." → **Reproducible lineage** from output to source data.

Data leakage / privacy

Sensitive data to a model. → **Governance, access control, deployment choices.**

Every risk maps to a layer. That's not a coincidence — **the stack is the risk-management strategy.**

A maturity model you can place yourself on

- 3 Proactive Management** AI acts ahead of the work; near-autonomous; human approval retained
- 2 Continuous Learning** System learns from feedback & adapts to law; agents run tasks for review
- 1 AI-Native Foundation** Copilots in daily use; grounded, with a real data foundation in place
- 0 Exploratory** PoCs, ungrounded chatbots, no evals — where most teams sit today

Most of the room is at **Level 0**. The next three years is climbing the three stages — **in order**.

Five moves that survive any model

1 · Fix the data foundation

It's the bottleneck and the highest-ROI work. Start with the data e-invoicing already gives you.

2 · Build an eval set

50 known-answer tax cases. Now you can measure any tool — and prove it works.

3 · Resolve ownership

Decide where tax-tech lives. Get Tax, Finance Systems & IT co-owning one stack.

4 · Pick one workflow

One high-volume, rules-based process. Ground it, instrument it, ship a copilot.

5 · Stay model-agnostic. Architect so you can swap the model in an afternoon, not a quarter.

Models will commoditize.
Infrastructure compounds.
**Build the stack, in order, and
earn autonomy with
evidence.**

Software → Infrastructure

Batch → Real-time

Headcount → Oversight

This thesis gets sharper with your numbers in it

Anonymous

Benchmark your automation, team structure, and AI stage against peers — no names attached.

Non-profit

Community-built, free, vendor-neutral. The dataset belongs to the field, not a SKU.

Equal access

Contribute once, see the whole picture. No premium tier, no gatekeeping.

If you run an in-house tax-tech function, you have a data point this field needs.

TRY IT LIVE

Contribute & compare

QR OFFLINE

taxbenchmark.ai

Generate a QR for this URL
before the talk and drop the
PNG in /presentation.

Scan to join the benchmark

taxbenchmark.ai

Free · anonymous · community-
driven. Your data calibrates the next
three years.

THANK YOU

Questions?

Let's talk about what's broken in your stack — that's where the next three years actually happen.

contact@taxbenchmark.ai · taxbenchmark.ai